Project- Honeypot and Canary Token Monitoring

Prepared by Mallikah Sharp

Introduction

Honeypots are designed as a decoy to attract malicious threat agents, using hacker tactics, techniques, and procedures (TTPs). It diverts attackers from the real systems and allows SOC teams to focus on protecting the real assets. The information gathered from the SOC teams help to improve the incident response and also helps in identifying and vulnerabilities. Canary tokens are like motion sensors for your networks, computers, and clouds. You can put them in folders, on your network devices and phones. You place them in a place where no one should be poking around, then you will get a clear alert if they are accessed. They are designed to look shiny so that it increases the likelihood that an attacker will open them.

Kali Linux set up and Honeypot Cowrie Installation

Run Command: *sudo apt-get update && sudo apt-get install git python3-virtualenv -y*

```
┌──(kali㉿kali)-[~]
└─$ sudo apt-get update && sudo apt-get install git python3-virtualenv -y
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
Reading package lists ... Done
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
git is already the newest version (1:2.47.2-0.1).
git set to manually installed.
python3-virtualenv is already the newest version (20.30.0+ds-1).
python3-virtualenv set to manually installed.
The following packages were automatically installed and are no longer required:
  icu-devtools libflac12t64 libfuse3-3 libgeos3.13.0 libglapi-mesa libicu-dev liblbfgsb0 libpoppler145 libpython
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 7 not upgraded.
```

Clone the official Cowrie git repository to your system.

Run Command: *git clone https://github.com/cowrie/cowrie.git*

```
┌──(kali㉿kali)-[~]
└─$ git clone https://github.com/cowrie/cowrie.git
Cloning into 'cowrie' ...
remote: Enumerating objects: 19005, done.
remote: Counting objects: 100% (153/153), done.
remote: Compressing objects: 100% (99/99), done.
remote: Total 19005 (delta 123), reused 54 (delta 54), pack-reused 18852 (from 3)
Receiving objects: 100% (19005/19005), 10.45 MiB | 14.26 MiB/s, done.
Resolving deltas: 100% (13348/13348), done.
```

Navigate the directory in the cowrie folder you just downloaded.

Run command: *cd cowrie*

```
┌──(kali㉿kali)-[~]
└─$ cd cowrie
```

Set up a Python Virtual environment for cowrie.

Run command: *virtualenv -p python3 cowrie-env*

```
┌──(kali㉿kali)-[~/cowrie]
└─$ virtualenv -p python3 cowrie-env
created virtual environment CPython3.13.2.final.0-64 in 1189ms
  creator CPython3Posix(dest=/home/kali/cowrie/cowrie-env, clear=False, no_vcs_ignore=False, global=False)
  seeder FromAppData(download=False, pip=bundle, via=copy, app_data_dir=/home/kali/.local/share/virtualenv)
    added seed packages: pip==25.0.1
  activators BashActivator,CShellActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator
```

Activate the virtual environment by running the

command: *source cowrie-env/bin/activate*

```
┌──(kali㉿kali)-[~/cowrie]
└─$ source cowrie-env/bin/activate
```

Install all of the necessary python packages.

Run Command: *pip install -r requirements.txt*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ pip install -r requirements.txt
```

Now we copy the default configuration file to create my own.

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

****Modify the configuration (optional)**:

- By default, Cowrie listens on port 2222. If you want to change it to the standard SSH port (22), open the configuration file in a text editor and modify the port setting: *nano etc/cowrie.cfg*
- Find the line that says listen_endpoints and change the port number to 22.

I will be using the default cowrie that is listening on port 2222.

Start the cowrie honey pot. Run command: *./bin/cowrie start*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ ./bin/cowrie start

Join the Cowrie community at: https://www.cowrie.org/slack/

Using activated Python virtual environment "/home/kali/cowrie/cowrie-env"
Starting cowrie: [twistd  --umask=0022 --pidfile=var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie
 ]...
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:105: CryptographyDeprec
ationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be
removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:112: CryptographyDeprec
ationWarning: TripleDES has been moved to cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be
removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
```

We can double check to see if our program is running using the

command: *./bin/cowrie status*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ ./bin/cowrie status
```

Check to see if cowrie is listening on the correct port.

Run command: *netstat -tuln | grep 2222*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ netstat -tuln | grep 2222
tcp        0      0 0.0.0.0:2222            0.0.0.0:*               LISTEN
```

Now simulate an attacker connecting to the honeypot.

Run Command: *ssh username@localhost -p 2222*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ ssh username@localhost -p 2222
```

Navigate to the log directory using the command: *cd var/log/cowrie*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ cd var/log/cowrie
```

To view the list of commands the attacker has attempted to run, use the tail command: *tail -f cowrie.log*

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie/var/log/cowrie]
└─$ tail -f cowrie.log
2025-04-26T04:45:44.318739Z [-] Ready to accept SSH connections
2025-04-26T05:31:06.017600Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 127.0.0.1:58134 (127.0.0.1:222
2) [session: 38611901330f]
2025-04-26T05:31:06.056465Z [HoneyPotSSHTransport,0,127.0.0.1] Remote SSH version: SSH-2.0-OpenSSH_9.9p2 Debian-
2
2025-04-26T05:31:06.076698Z [HoneyPotSSHTransport,0,127.0.0.1] SSH client hassh fingerprint: 0babd4b68a5f3757987
be75fe35ad60a
2025-04-26T05:31:06.081875Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] kex alg=b'curve25519-sha256' key a
lg=b'ssh-ed25519'
2025-04-26T05:31:06.082897Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] outgoing: b'aes128-ctr' b'hmac-sha
2-256' b'none'
2025-04-26T05:31:06.083523Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] incoming: b'aes128-ctr' b'hmac-sha
2-256' b'none'
2025-04-26T05:33:06.148552Z [-] Timeout reached in HoneyPotSSHTransport
2025-04-26T05:33:06.188819Z [cowrie.ssh.transport.HoneyPotSSHTransport#info] connection lost
2025-04-26T05:33:06.191091Z [HoneyPotSSHTransport,0,127.0.0.1] Connection lost after 120.1 seconds
```

Analyse the attack data. This can be helpful to understand the attack patterns and malicious behaviours.

Now you can stop cowrie with the command: *cd ~/cowrie ./bin/cowrie stop*

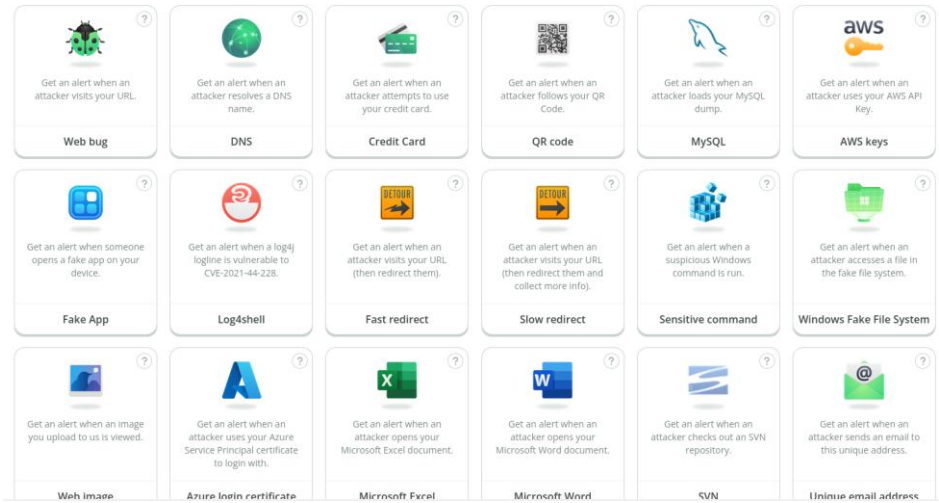
```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie/var/log/cowrie]
└─$ cd ~/cowrie
./bin/cowrie stop

Stopping cowrie ...
```

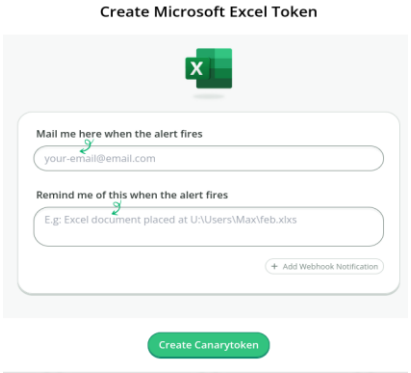
Canary Tokens

Navigate to the Canary Tokens website: https://canarytokens.org/nest/

You can see many options to create your canary token. You can choose from different types of files like, word, pdf, excel, as well as Azure login certificate or Windows folder.



Select the token you wish to configure and set up. I chose an excel spreadsheet. Choose the email address where you would like the notification to be sent. Download the Canary token.

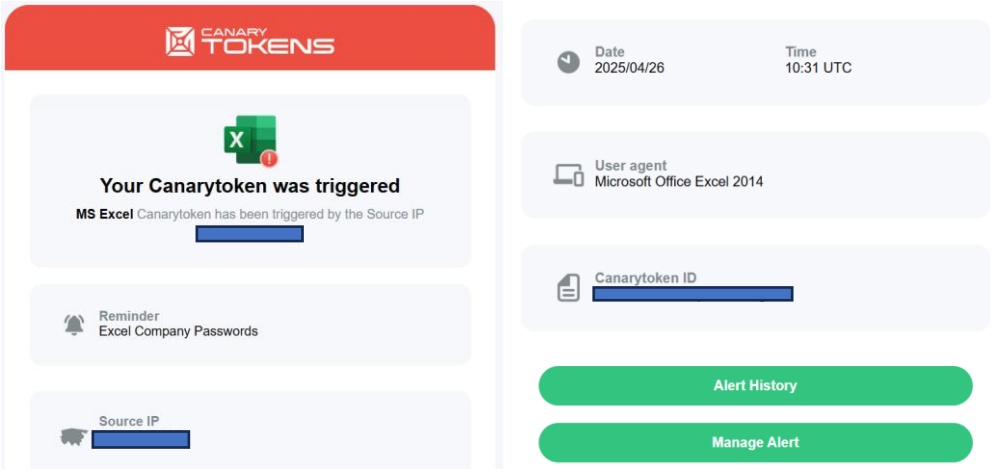


Create your Honey Pot Scenario

When the token is created, you will receive confirmation on your screen. It will say, "New Token Created!"

Click the green button to download the file. Then, change the name of the file to trick the threat agents into opening it believing there is sensitive information. Place it in a folder that is relevant to the file name. Place some fake data inside the file. During my training, I open the file myself. When I open the empty file, there will not be any data, but I will receive a notification on the specified email address that the canary token has been activated. Next, I go to the email account to see specific details. The email will include the following details:

- Source IP of the person who accessed the file
- User agent
- Timestamp



Conclusion

In this project, I learned that canary tokens are very useful tools for SOC teams to help detect unauthorized system access by providing alerts in real-time. It is proactive and gives me the ability to detect advanced threats and APTs using deception tools. The tokens do not contain any real data as their sole purpose is to lure threat agents into using them. By monitoring how attackers interact with tokens will help to improve incident response.